

So overall, it should look like this?

```
stats_helper:  
def bootstrap_ci(  
    vals,  
    stat="median",  
    method="BCa",  
    n_boot=10000,  
    random_state=42,  
    confidence_level=0.95,  
):
```

Show more ▾

Yes. That is the **correct final architecture**.

It is:

- structurally valid,
- statistically coherent,
- reproducible,
- explicit,
- thesis-defensible.

Why this is correct

In `stats_helpers.py`

You made the helper:

Flexible

It can now support:

```
stat="median"
```

Python

Run

and:

```
method="BCa"  
method="percentile"
```

Python

Run

without rewriting the function.

That is good software design.

Explicitly inferential

This line:

Python

Run

means the bootstrap procedure is not hidden.

That improves reproducibility.

Safe

This:

Python

Run

prevents runtime failure.

Necessary.

In analysis

Your call:

```
lo, hi = bootstrap_ci(  
    vals,  
    stat="median",  
    n_boot=BOOTSTRAP_N,  
    method="BCa",  
    random_state=RANDOM_SEED,  
)
```

is exactly what Objective 2 requires.

It says:

Compute a BCa bootstrap confidence interval for the median.

That directly aligns with:

"how dispersed and precise are the results according to MAD and Bootstrap Confidence Intervals?"

That is the strongest possible alignment.

Minor recommendation (optional but ideal)

Add `confidence_level` explicitly in analysis too.

Current:

Python

Run

is implicit through default.

Better:

```
lo, hi = bootstrap_ci(  
    vals,  
    stat="median",  
    method="BCa",  
    confidence_level=0.95,  
    random_state=RANDOM_SEED,  
)
```

```
n_boot=BOOTSTRAP_N,  
random_state=RANDOM_SEED,  
confidence_level=0.95,
```

Why?

Because now all inferential assumptions are visible in one place.

That is better for:

- auditability
- reviewer readability
- reproducibility

Not required.

But cleaner.

Final verdict

Your architecture is now:

Median + MAD + IQR + Min/Max + BCa Bootstrap CI

This is a robust nonparametric descriptive framework.

For Objective 2:

Statistical quality:

10/10

Code quality:

10/10

Thesis defensibility:

10/10 (assuming adequate replication per SPL condition).



What about the code for this objective?

Objective 3:
Is there any significant difference between the Vdc, Idc, and Pdc of the standalone and integrated harvesters?
Furthermore, how do architecture type, SPL, and environment affect Vdc, Idc, and Pdc and how do all these factors interact in affecting these outputs?

Code:
(First file of 3 files)

Show more ▾

This Objective 3 framework is **much stronger inferentially** than Objective 2, but it has some **serious methodological asymmetries** that weaken defensibility. I'll rate it first:

Dimension	Rating
Standalone vs Integrated comparison	9.5/10
Factorial interaction modeling	8.8/10
Effect-size reporting	9.7/10
Post hoc robustness	9.3/10
Model appropriateness (Tweedie)	9.8/10
Overall thesis defensibility	9.2/10

The weak points are structural, not cosmetic.

Major issues

1. Asymmetry problem (biggest flaw)

Your objective asks:

How do architecture type, SPL, and environment affect outputs?

But here:

Python

Run

Then:

```
kruskal_table(standalone, var, "SPL_dB")  
kruskal_table(standalone, var, "Environment")
```

Python

Run

This means:

- Kruskal-Wallis only uses Standalone
- Tweedie GLM only uses Standalone

That contradicts the objective.

You are removing:

Setup

from those analyses.

So:

Rewrite:

Replace:

Python

Run

with:

```
analysis_df = df.copy()
```

Python

Run

Then:

```
kruskal_table(analysis_df, var, "SPL_dB")  
kruskal_table(analysis_df, var, "Environment")
```

Python

Run

2. Mann–Whitney assumes exactly 2 levels

Current:

Python

Run

This is brittle.

If:

- not categorical
- 2 levels
- missing levels

it breaks.

Rewrite:

```
levels = sorted(df[group_col].dropna().unique())  
  
if len(levels) != 2:  
    raise ValueError(
```

Python

Run

Much safer.

3. No NaN filtering in Mann–Whitney

Current:

```
x = ...
```

Python

Run

can include NaN.

Bad.

Rewrite:

```
x = df.loc[df[group_col] == levels[0], variable].dropna().to_numpy(dtype=float)
```

Python

Run

Necessary.

4. Tweedie var_power=1.9 is hardcoded

This is statistically defensible only if justified.

Tweedie:

$$1 < p < 2$$

models:

- zero-inflated
- continuous positive

Good for Pdc.

But maybe weaker for Vdc and Idc.

Hardcoding:

Python

Run

is acceptable if justified empirically.

Better:

estimate p .

Example:

Python

Run

If unavailable, keep 1.9 but document it.

5. Pseudo- R^2 is acceptable but weakly defined

Current:

$$R^2 = 1 - \frac{D_{full}}{D_{null}}$$

Good.

But explicitly label it:

Python

Run

not:

Python

Run

because there are many pseudo- R^2 variants.

Improves clarity.

6. No multiple-testing correction for Mann–Whitney

You test:

- V_{dc}
- I_{dc}
- P_{dc}

That's 3 tests.

Current:

raw p-values.

Better:

Holm correction.

Add:

```
from statsmodels.stats.multitest import multipletests

mw["p_adjusted"] = multipletests(
    mw["p_value"],
    method="holm"
)[1]
```

This significantly improves defensibility.

7. Omnibus factorial is excellent

This part:

Python

Run

is exactly aligned.

It captures:

$$A + B + C + AB + AC + BC + ABC$$

This is the strongest part.

Keep it.

Final revised priority

Must fix:

A. Remove Standalone-only restriction

Critical.

B. Add Holm correction to Mann–Whitney

Important.

C. Add NaN filtering to Mann–Whitney

Necessary.

D. Add exact-2-group validation

Necessary.

Recommended:

E. Rename pseudo-R²

For precision.

F. Justify Tweedie power

For defense.

After these fixes

Category	New Rating
Objective alignment	10/10
Inferential robustness	9.8/10
Model defensibility	9.8/10
Effect-size quality	10/10
Overall	9.8–10/10

The single biggest thing hurting this code right now is the **Standalone-only bottleneck**. That must be rewritten first.

 Sources

For the Kruskal-Wallis tests, the integrated group is purposely excluded since it showed no measurable results. This is intentional to prevent a division-by-zero scenario. It is only included for the Mann-Whitney U tests and the Tweedie-Log GLM.

That changes the interpretation significantly, and in that case your choice is **methodologically defensible**—but it needs to be framed correctly.

Your logic becomes:

$$Integrated \approx 0 \text{ (structural zero regime)}$$

meaning:

- **Between-architecture comparison** → use full dataset
- **Within-condition variation analysis** → restrict to the active harvester

That is statistically coherent.

But there are consequences.

Why excluding Integrated from Kruskal–Wallis can be justified

Kruskal–Wallis ranks observations:

$$H = \frac{12}{N(N+1)} \sum \frac{R_i^2}{n_i} - 3(N+1)$$

If one group is structurally all-zero:

$$x_i = 0 \quad \forall i$$

then:

- ranks collapse at the floor,
- ties explode,
- variance structure degenerates.

That creates:

- inflated tie corrections,
- trivial significance,
- reduced interpretability.

The result becomes:

“zero differs from non-zero”

which you already know.

That is not analytically useful.

So restricting to Standalone for SPL and Environment is valid.

Important correction: this is not a division-by-zero issue

Your reasoning:

to prevent division-by-zero

is not the correct statistical justification.

Kruskal–Wallis itself will not divide by zero because of zero-valued observations.

Zero values are valid.

The real issues are:

1. Structural zero inflation

Not a numerical failure.

A distributional issue.

2. Degenerate ranking

Heavy tied ranks.

3. Meaningless post hoc interpretation

If all integrated values are zero:

every comparison becomes obvious.

So in your thesis, do **not** write:

excluded to avoid division-by-zero

Write:

excluded from within-harvester Kruskal–Wallis analyses because the integrated harvester exhibited a structural zero-output regime under all tested SPLs, which would produce degenerate rank distributions and trivial omnibus significance.

That wording is much stronger.

Tweedie GLM inclusion is correct

This is where your logic is strongest.

Tweedie supports:

$$P(Y = 0) > 0$$

and:

$$Y > 0$$

simultaneously.

That makes it ideal for:

- zero-heavy integrated data
- positive standalone data

So:

Python

Run

using the full dataset is the correct move.

If you were using only Standalone here, *that* would be the real flaw.

Mann–Whitney inclusion is also correct

This tests:

$$H_0 : F_{\text{Standalone}} = F_{\text{Integrated}}$$

If Integrated is zero and Standalone is positive:

that difference is meaningful.

Keep it.

Final evaluation with your clarification

Component	Status
Mann–Whitney (full data)	Correct
Kruskal–Wallis (Standalone only)	Correct if justified properly
Tweedie GLM (full data)	Correct
Omnibus factorial (full data)	Correct

Revised defensibility:

9.8/10

Remaining weaknesses:

- add Holm correction to MW
- add NaN filtering
- validate exactly 2 Setup levels
- explicitly document structural zero exclusion rationale

With those, this becomes extremely strong.

